Process and Decision Documentation

Group: 10B
Assignment: A2 - Mid-Term Game
Disability Topic: Traumatic Brain Injury (TBI)
Updated: March 9, 2026

---

1. Process Overview Visualization

Timeline

Phase	Early (Mar 3)	Mid (Mar 4–5)	Late (Mar 9)
Focus	Research + framing	Playable build + rubric completion	Refactor + presentation polish
Jimin	Project scaffold, repo setup	Integrated large code pass, verified feature coverage	Converted game to stage-based flow, rewrote title flow and start UI
Kaiyang	Mechanic concepting	Tuned memory + overload behavior	Group focus for showcase — checked whether pacing reads clearly
Catarina	Level progression ideas	Layout balancing + checkpoints	Group focus for showcase — checked whether pacing reads clearly
Annora	UX/accessibility research	Visual polish + readability	Group focus for showcase — checked whether pacing reads clearly

Key Pivots

1. **Skeleton → full empathy game** — From a simple scaffold to a feature-complete game with sound, feedback, and accessibility support.
2. **Continuous map → staged progression** — One long multi-room map felt less readable than three clearly separated stages with escalating strain.
3. **Text-heavy title → clean menu** — Replaced dense title text with a How to Play overlay and stronger visual hierarchy.

---

2. Role-Based Contributions

Member	Primary Role	Shadow Role	Key Contributions

Member	Primary Role	Shadow Role	Key Contributions
Jimin Kim	Project Setup / Core Game Loop	Game Enhancement	Set up the repository and initial p5.js project structure. Implemented and later revised the core state machine. Used Claude Code and Codex to scaffold the first playable version, then refactored the game into a 3-stage structure, added stage transitions, polished the title screen, and updated the documentation to match the current build.
Kaiyang Sun	TBI Mechanics Design & Implementation	Gameplay Systems	Helped define which TBI-related experiences should become mechanics. Implemented Memory Fade and Sensory Overload in the earlier prototype, then tuned mechanic behavior so the stages remained readable while still communicating strain.
Catarina Jin	Level Design	Gameplay Testing	Designed progression flow, task placement, and checkpoint logic. Identified confusion points during testing and helped rebalance routes so navigation pressure increased over time without becoming random.
Annora Jo	UX/UI Design & Art/Sound	Documentation	Shaped the game's visual tone and readability, including character expression changes, UI clarity, accessibility thinking, and the distinction between normal and Low Sensory Mode. Contributed to keeping the presentation respectful and grounded for a TBI-related showcase context.

3. Key Design Decisions

3.1 Current Project Structure

The current build uses a small, readable p5.js structure:

- `index.html` loads the local p5.js library and the game sketch
- `sketch.js` contains the game loop, stage data, mechanics, UI, and procedural art/audio
- `style.css` styles the page and canvas container
- `libraries/p5.min.js` is stored locally as required

We intentionally kept the code in a single `sketch.js` file because the course level is introductory and the team wanted the logic to remain readable during showcase explanations.

3.2 State Machine and Stage System

The original prototype used a simpler start/play/win/lose loop and a continuous map. The current build uses five states:

- `STATE_START`
- `STATE_PLAY`

- `STATE_STAGE_TRANSITION`
- `STATE_WIN`
- `STATE_LOSE`

This change mattered because the earlier one-map version flattened the emotional progression. Refactoring into a data-driven stage system made the day easier to read as a sequence:

- Stage 1: Home
- Stage 2: Outside / Store
- Stage 3: Workplace / Way Home

Each stage now loads its own walls, zones, decorations, checkpoints, hints, objective text, and overload pacing through `createStages()`, `initGame()`, and `loadStage()`. This was a deliberate design decision to communicate cumulative strain more clearly.

3.3 Representing TBI Through Mechanics

The final build combines several mechanics so the player feels layered cognitive effort rather than a single abstract difficulty system:

- **Memory Fade**: the objective disappears and must be recalled with `M`
- **Sensory Overload**: overload rises constantly and spikes in red noise zones
- **Cognitive Fatigue**: movement slows as overload rises
- **Intrusive Distractions**: brief flashes appear under higher overload
- **Attention Drift**: later-stage movement becomes less precise at high strain
- **Emotional Frustration Messages**: internal thoughts appear when overload is high
- **Fading Awareness**: distant task markers become harder to notice later in the game

The design goal was not to simulate every aspect of TBI literally. Instead, we used mechanics that are readable in a short p5.js project but still grounded in reported experiences such as memory difficulty, fatigue, sensory pressure, and frustration [1][2][3].

3.4 Stage Progression as an Empathy Decision

The most important late-project change was replacing the continuous five-room route with a staged progression.

What changed:
The older build showed the day as one long left-to-right map. The current build separates the experience into three stages with transition screens and carry-over fatigue.

Why it changed:
The continuous map communicated space, but not escalation very well. Players were seeing all areas as part of one puzzle rather than feeling a day becoming harder. The staged version lets us pace mild strain, then stronger sensory pressure, then cumulative fatigue and frustration.

Who was involved:
Jimin led the implementation with Codex support. The change also followed earlier group discussion about making the emotional progression clearer for showcase audiences.

3.5 Title Screen and Instruction Design

The title screen was also simplified late in the process.

What changed:
Controls are no longer always visible on the main title page. The start screen now focuses on the title, subtitle, primary start action, and a smaller **How to Play** button that opens a centered overlay.

Why it changed:
The earlier title layout looked polished but still felt too much like a presentation slide. Moving instructions into an overlay reduced clutter and made the first screen feel more like a game menu while staying readable and calm.

Who was involved:
Jimin implemented the layout changes through several Codex-assisted iterations, while the overall visual tone remained aligned with the team's established UX direction.

3.6 Accessibility and Player Control

Low Sensory Mode remained a central accessibility feature throughout development. It removes or reduces:

- flashes and distraction visuals
- glow and particles
- vignette, haze, and shake
- detailed textures and pulsing filled zones

We also added R to return to the title screen during play, win, and lose states. This is a small but important usability decision because players should be able to reset without refreshing the page.

3.7 Ethical and Representational Framing

Because the project topic is TBI and people with TBI may be present at the showcase, we treated tone as a design decision rather than just writing style.

The game avoids:

- horror framing
- pity-based messaging
- random enemies or chaotic punishment systems

Instead, it frames difficulty around effort, pacing, overload, and recovery. Calm Zones remain meaningful, the win screen acknowledges effort rather than "beating" TBI, and even the lose screen is written to avoid treating overwhelm as failure.

3.8 Playtesting and Iteration

The most important observations that affected the current build were:

1. Players needed clearer pacing and stronger stage-to-stage escalation.
2. The title screen needed less information visible at once.
3. Calm Zones and memory recall had to stay readable without adding heavy tutorial text.

These observations directly led to the stage refactor, the transition screens, and the How to Play overlay.

---

4. GenAI Use Documentation

4.1 Task: Project Structure Setup and Core Game Loop

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 3, 2026
Task Description	Setting up the project folder structure, creating the required files, and implementing the initial state-based p5.js game loop.
How GenAI Was Used	Claude Code was given the assignment requirements and asked to scaffold the folder structure and initial p5.js files.
What Was Modified After Generation	The scaffold became the base for later human revisions, including stage logic, UI changes, and updated documentation.

4.2 Task: Git Repository Setup Assistance

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 3, 2026
Task Description	Troubleshooting git push errors when publishing to GitHub.
How GenAI Was Used	Claude Code identified a branch naming mismatch and suggested renaming the local branch before pushing.
What Was Modified After Generation	No code changes. Commands were followed manually.

4.3 Task: Early TBI Mechanics Brainstorming

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 3, 2026
Task Description	Brainstorming beginner-friendly ways to represent TBI-related strain through gameplay.
How GenAI Was Used	ChatGPT suggested simple versions of Memory Fade and Sensory Overload using timers, meters, and calm/recovery spaces.
What Was Modified After Generation	The mechanic ideas were simplified and then integrated into the team's own structure and pacing decisions.

4.4 Task: Integrating Mechanics into the Existing Game Structure

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 3, 2026
Task Description	Integrating the gameplay mechanics into the existing game structure that uses start, play, win, and lose states.
How GenAI Was Used	ChatGPT helped identify where different parts of the mechanics should be placed in the code structure. Variables were placed in the game variables section, initialization logic in `resetGame()`, updates in `updateGame()`, and rendering in `drawPlayScreen()`.
What Was Modified After Generation	The structure was adapted slightly to fit the code that had already been created earlier in the project by another group member.

4.5 Task: Level Design and Progression Planning

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 4, 2026
Task Description	Planning how players should learn the mechanics before facing the full challenge.
How GenAI Was Used	ChatGPT suggested gradual progression, checkpoints, and clearer guidance for first-time players.
What Was Modified After Generation	The team manually adjusted layout, task placement, and pacing to fit the actual code and playtesting feedback.

4.6 Task: UX Research for TBI-Sensitive Design

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5.2)
Date	March 4, 2026
Task Description	Researching design considerations for players with TBI.
How GenAI Was Used	ChatGPT summarized considerations such as reducing cognitive load, minimizing sensory overload, supporting resuming, and keeping feedback clear.
What Was Modified After Generation	The team applied those ideas selectively through Low Sensory Mode, calmer UI framing, and readable feedback rather than copying them directly.

4.7 Task: Rubric Completion Pass

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 5, 2026
Task Description	Evaluating the game against the rubric and implementing missing features.
How GenAI Was Used	Claude Code was asked to identify rubric gaps and then implement missing systems such as audio feedback, stronger level structure, screen effects, and citations.
What Was Modified After Generation	The team further tuned Low Sensory Mode and continued revising the structure after the rubric-completion pass.

4.8 Task: Facial Expression Fix

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 5, 2026
Task Description	Fixing the player face so high-overload expressions looked distressed rather than cheerful.
How GenAI Was Used	ChatGPT explained the p5.js `arc()` direction issue and suggested a four-stage expression system.
What Was Modified After Generation	Eyebrow and mouth positions were manually adjusted after generation to improve the final expression read.

4.9 Task: Input Debugging

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 5, 2026
Task Description	Fixing non-responsive `M` and `L` key inputs.
How GenAI Was Used	Claude Code identified unreliable mixed use of `key` and `keyCode` and replaced string checks with numeric keyCode checks.
What Was Modified After Generation	No further modification was needed after verification.

4.10 Task: Low Sensory Mode Contrast Pass

Field	Details

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 5, 2026
Task Description	Making the difference between normal mode and Low Sensory Mode more legible.
How GenAI Was Used	Claude Code was prompted to strengthen visual simplification by removing more effects and flattening the scene.
What Was Modified After Generation	Later UI refinements preserved the same accessibility goal while simplifying the title screen presentation.

4.11 Task: Stage-Based Refactor

Field	Details
GenAI Tool Used	Codex (OpenAI GPT-5)
Date	March 9, 2026
Task Description	Refactoring the game from one continuous map into a true 3-stage progression while preserving the mechanics, art style, and emotional tone.
How GenAI Was Used	Codex was given the current `sketch.js` and asked to preserve as much code as possible while replacing room-based flow with stage-based progression, stage transitions, stage-specific data, and a final-stage win condition.
What Was Modified After Generation	Jimin followed with several UI/layout refinement prompts to improve the title screen after the structural refactor was complete.

4.12 Task: Title Screen Simplification and How to Play Overlay

Field	Details
GenAI Tool Used	Codex (OpenAI GPT-5)
Date	March 9, 2026
Task Description	Simplifying the start screen, moving controls into a modal overlay, and improving title-state navigation.
How GenAI Was Used	Codex was prompted to reduce visible text, create a clickable How to Play modal, add R to return to title, and preserve the existing dark visual language.
What Was Modified After Generation	Jimin then iterated on button sizing, spacing, and title alignment through additional follow-up prompts.

4.13 Task: Start Screen Layout Polish

Field	Details
GenAI Tool Used	Codex (OpenAI GPT-5)
Date	March 9, 2026
Task Description	Refining the proportions and alignment of the title screen without redesigning the whole screen.
How GenAI Was Used	Codex adjusted button sizes, removed extra helper text, centered text blocks within the panel, and finally aligned the title and subtitle to the faded inner rectangle rather than the full navy panel.
What Was Modified After Generation	These were iterative polish passes guided by Jimin's visual feedback rather than one single generated output.

---

5. Appendix - Conversation Logs

The logs below are written in the same concise transcript style used earlier in this document. The March 9 entries were all part of one continuous Jimin/Codex iteration session on the current build. Later prompts refined the output of earlier prompts rather than starting from scratch each time.

5.1 Claude Code - Project Setup (Jimin, March 3)

User (Jimin):

> Please read this document [A2 assignment PDF]. My role is to set up the project structure and core game loop (start/play/win/lose). The assigned disability for our group is Traumatic Brain Injury. Please set up the appropriate files into this directory.

Claude Code:

> Created the initial project structure including `index.html`, `sketch.js`, `style.css`, `jsconfig.json`, `README.md`, and the local p5.js library, along with an initial 4-state playable skeleton.

5.2 ChatGPT - Mechanics Implementation (Kaiyang, March 3)

User (Kaiyang):

> [A2 assignment PDF] I have a p5.js game skeleton with start, play, win, and lose states. I need to implement gameplay mechanics that represent aspects of Traumatic Brain Injury, but the code needs to remain simple because this is an introductory programming course.

ChatGPT:

> Suggested implementing a "Memory Fade" mechanic using a timer that hides the objective text after a short period of time, along with a key press that allows the player to recall the objective temporarily.

User (Kaiyang):

> I also want to represent sensory overload in the game without using complex visual effects.

ChatGPT:

> Recommended creating an overload meter that increases over time and placing a calm zone in the level where the overload value decreases when the player enters the area.

User (Kaiyang):

> The code should avoid advanced math functions.

ChatGPT:

> Provided simplified logic using basic variables, timers, conditional statements, and distance checks to implement the mechanics using beginner-level p5.js code.

5.3 ChatGPT - Level Design (Catarina, March 4)

User (Catarina):

> Our group already implemented the mechanics for a p5.js game about Traumatic Brain Injury. My role is the Level Designer. I need to design a level that teaches the mechanics first and then combines them so that first-time players can understand how the game works. What level structure would work for this?

ChatGPT:

> Suggested dividing the level into three phases. Phase 1 introduces the objective and basic movement by placing a few stars close to the player so they can learn the goal without pressure. Phase 2 introduces the Memory Fade mechanic, where the objective disappears after a short time and players can press a key to recall it. Phase 3 combines both mechanics by placing the remaining stars farther away while the overload meter continues increasing.

User (Catarina):

> I also need to make sure players do not get stuck or lose too quickly during playtesting. What design changes could help make the level easier to understand?

ChatGPT:

> Recommended adding visual cues and recovery systems to help first-time players.

5.4 ChatGPT - UX Research (Annora, March 4)

User (Annora):

> what are some aspects to know when designing for a user with Trauma Brain Injury?

ChatGPT:

> Explained the common symptoms that a TBI user might encounter, and suggested to make the experience calm, predictable and easy to resume. Listed some aspects to consider: Reduce cognitive load, Support memory and "resuming", Minimize sensory overload, Make timing forgiving, Clear feedback and error recovery, Consistency and predictability, Accessibility beyond cognition (often relevant with TBI), Give control to the user.

5.5 Claude Code - Rubric Review and Major Enhancement Pass (Jimin, March 5)

User (Jimin):

> Based on this A2 file, can you tell me does this code game satisfies the rubric fully? Our group topic is Traumatic Brain Injury.

Claude Code:

> Evaluated the game against all 8 rubric criteria and identified critical gaps: no sound/audio, no references or citations in code or README, process doc needs PDF format, README sections incomplete, no visual assets (basic shapes only), level feels empty/minimal. Provided a detailed breakdown by criterion with specific issues and priorities.

User (Jimin):

> Then can you change all the features first to solve all of the issues and satisfies all of the rubric and requirements please. Also add sound and assets like images?

Claude Code:

> Rewrote sketch.js with comprehensive enhancements: wall obstacles creating three connected rooms, stimulus zones, Web Audio API sound system (7 sound types + dynamic ambient drone), particle effects, cognitive fatigue speed reduction, tunnel vision vignette, screen shake, floor tiles, decorations, narrative framing on all screens, Low Sensory Mode. Updated index.html with meta tags and accessibility attributes. Created assets directory structure.

5.6 Claude Code - Input Fix (Jimin, March 5)

User (Jimin):

> The M and L keyboards are not working. I mean when I pressed M or L, nothing happened.

Claude Code:

> Identified that p5.js's `key` variable is unreliable when used alongside `keyCode`, and replaced the string comparisons (`key === "m"`) with numeric keyCode checks (`keyCode === 77` for M, `keyCode === 76` for L).

5.7 Claude Code - Low Sensory Mode Contrast Pass (Jimin, March 5)

User (Jimin):

> Make more difference between On mode of the Low sensory Mode and Off mode.

Claude Code:

> Enhanced Low Sensory Mode with dramatic visual differences: lighter background, hidden floor tiles, outlined-only stimulus zones, flat solid walls, hidden particles, simple circle stars without glow, simplified calm zone, persistent on-screen indicator badge.

User (Jimin):

> For the decorations, when the low sensory mode, do not make it as hidden entirely just make it simple solid rectangle please.

Claude Code:

> Changed decorations in Low Sensory Mode from hidden entirely to simple solid rectangles without shadow or highlight effects.

5.8 ChatGPT - Player Facial Expressions (Jimin, March 5)

User (Jimin):

> this is the sketch code, and when the character in the game overload gets high, suddenly character in the game smile. why?

ChatGPT:

> In your code, the "smile" happens because of this line: `arc(px, py - 5, 6, 4, 0, PI);`. In p5.js, `arc(x, y, w, h, start, stop)` draws part of an oval between two angles. Drawing from 0 to PI often creates the bottom half of an arc, which looks like a smiling mouth. To make it look like a frown instead, flip the arc angles: `arc(px, py - 5, 6, 4, PI, TWO_PI);`

User (Jimin):

> please change the version of faces in steps of neutral -> anxious -> overloaded.

ChatGPT:

> Provided a multi-stage expression system to paste inside `drawPlayer()`, replacing the old single `if (overload > 70)` block with four expression stages using combinations of `line()`, `ellipse()`, `arc()`, and `rect()` for the mouth, and angled lines for eyebrows.

User (Jimin):

> `line(px - 3, py - 3, px + 3, py - 3);` how to move the mouth up

ChatGPT:

> The mouth's vertical position is controlled by the `py - 3` part. To move the mouth up, make the y-value smaller (subtract more from py). Recommended using a variable: `let mouthY = py - 6;` so it's easy to adjust.

User (Jimin):

> what is the syntax of line

ChatGPT:

> In p5.js, the syntax for `line()` is: `line(x1, y1, x2, y2);` where (x1, y1) is the starting point and (x2, y2) is the ending point.

User (Jimin):

> move the eyebrows down a little bit

ChatGPT:

> To move the eyebrows down, increase the y-values (bigger y = lower on screen). Suggested changing `py - 14.8` to `py - 12.8` and `py - 16.2` to `py - 14.2`. Recommended using an offset variable: `let browDown = 2;`

User (Jimin):

> move the mouth and teeth up a little bit

ChatGPT:

> To move the mouth + teeth up, decrease the y-values. Suggested using one offset variable: `let mouthUp = 2;` applied to all mouth and teeth coordinates.

5.9 Codex - Stage-Based Refactor (Jimin, March 9)

User (Jimin):

> Please read this code and current files and modify all of below. Important: do not just describe what to do. Actually rewrite the code into a working stage-based version using my current code as the base.
>
> Please refactor my current p5.js game code from one continuous 5-room map into a true stage-based progression with separate levels/scenes, while preserving as much of the current code, mechanics, art style, and emotional tone as possible.
>
> Important project context: This is a class project about Traumatic Brain Injury (TBI). The game should help non-disabled players feel some of the cognitive and sensory strain associated with TBI during ordinary tasks. The audience at showcase may include people with TBI, so the design must remain respectful, grounded, empathetic, and not pity-based or exaggerated.
>
> Current problem: Right now the game is built as one long continuous map with five rooms shown at once: Home, Outside, Store, Workplace, Way Home.

> I want this changed so the game progresses through separate stages instead of one giant map. The new structure should make the emotional and mechanical progression clearer:
>
> - Stage 1 = mild cognitive strain
> - Stage 2 = sensory pressure and memory difficulty
> - Stage 3 = cumulative fatigue, overload, frustration, and harder navigation
>
> What I want you to do:
>
> 1. Refactor the current code into a staged game structure.
> 2. Replace the current one-map flow with 3 separate stages/scenes.
> 3. Keep the same core mechanics:
> - Memory Fade
> - Sensory Overload
> - Cognitive Fatigue
> - Intrusive Distractions
> - Attention Drift
> - Emotional Frustration messages
> - Fading Awareness
> - Low Sensory Mode
> 4. Preserve the current visual style and drawing-based approach in p5.js.
> 5. Reuse as much of the existing code as possible instead of rewriting everything from scratch.
>
> Technical refactor requirements:
>
> - Create a stage system using data-driven configuration if possible.
> - Add something like `let currentStage = 0;`
> - Replace `initLevel()` with something like `initGame()` and `loadStage(stageIndex)`
> - Add a stage transition screen if helpful
> - Update the win condition so the player wins after finishing the final stage
> - Use `STATE_STAGE_TRANSITION`
> - Update the HUD to show stage name and stage progress
> - Remove old room-based logic if it is no longer needed
>
> Refactor strategy I prefer:
>
> - Minimal but clean restructuring
> - Preserve current naming where reasonable
> - Avoid overengineering
> - Keep it beginner-student readable
> - Use plain JavaScript and p5.js only
>
> Please return your answer in this order:
>
> 1. A short implementation plan
> 2. The new stage data structure
> 3. The specific functions I should replace or add
> 4. The full updated code
> 5. A brief explanation of how the new structure better supports the TBI experience goal

Codex:

> Returned the requested 5-part response structure and rewrote `sketch.js` into a 3-stage data-driven system.
>
> The implementation:
>
> - added `STATE_STAGE_TRANSITION`
> - introduced `createStages()`, `initGame()`, `loadStage()`, `updateStageProgression()`, `drawStage()`, and `drawStageTransitionScreen()`
> - replaced the old continuous room progression with separate Stage 1, Stage 2, and Stage 3 data
> - preserved the existing mechanics, particle system, audio, player drawing, overload bar, calm zones, vignette, and start/win/lose screens
> - changed the win condition so the player wins after finishing the final stage rather than clearing one giant map
>
> The explanation emphasized that the new staged structure better communicates cumulative strain across an ordinary day rather than treating the experience as one flat challenge.

5.10 Codex - Polished Start Screen and Return-to-Title Flow (Jimin, March 9)

User (Jimin):

> Please redesign the start screen of my p5.js game so it feels more polished, immersive, emotionally grounded, and visually aligned with the game's TBI empathy goal.
>
> Context:
>
> - The game is called "Fragmented"
> - It is a respectful, empathy-driven game about TBI
> - The audience at showcase may include people with TBI
> - The presentation must remain respectful, readable, calm, and not overly dramatic
>
> Current issues:
>
> - It still feels too static
> - There is too much text on screen
> - The controls panel is slightly overloaded
> - The Start button should feel more visually important
> - Low Sensory Mode should be clearer as an accessibility option
>
> What I want:
>
> 1. Keep the same overall dark, atmospheric tone
> 2. Keep the title "Fragmented" as the main focal point
> 3. Make the layout feel more like a strong game start screen and less like a class presentation slide
> 4. Reduce text density and improve hierarchy
> 5. Make the start action feel stronger and more interactive

> 6. Add support for an `R` key that returns the player to the title screen during gameplay
>
> Also add:
>
> - a short subtitle
> - a shorter intro paragraph
> - an optional progression strip
> - a simpler controls card
> - a more prominent start button
> - a clearer Low Sensory Mode line
> - a quiet secondary support line for returning to title during play
>
> Please give:
>
> 1. a short design explanation
> 2. the updated start screen code
> 3. the keyboard handling changes needed for the `R` key

Codex:

> Redesigned the start screen into a more atmospheric title layout with:
>
> - a stronger gold title focal point
> - shorter and more emotionally direct intro text
> - a journey/progression strip
> - a simplified controls section
> - a more prominent `Press ENTER to Start` button
> - clearer Low Sensory Mode wording
>
> Also added `returnToTitleScreen()` and updated `keyPressed()` so pressing `R` during play, win, or lose screens cleanly returns the game to `STATE_START`.

5.11 Codex - Simplified Start Screen with How to Play Overlay (Jimin, March 9)

User (Jimin):

> Please simplify and redesign my game's start screen UI so it feels cleaner, less cluttered, and more polished.
>
> Main design goal:
>
> I want the start screen to be much simpler. The main start page should only show:
>
> - Title: Fragmented
> - A short subtitle or very short intro
> - The main start button
> - A smaller `How to Play` button
> - A small Low Sensory Mode line below the main button
>
> The controls and key explanations should only appear after the player presses `How to Play`.

> Please implement:
>
> - `showHowToPlay = false`
> - a clickable `How to Play` button on the start screen
> - a `drawHowToPlayOverlay()` function
> - keyboard handling for opening/closing the overlay
> - `mousePressed()` handling for the button
> - the overlay should work only on the start screen
> - keep the code in a single p5.js file
>
> Please provide:
>
> 1. a short design explanation
> 2. the updated start screen code
> 3. the new variables/state needed
> 4. the new `drawHowToPlayOverlay()` code
> 5. the keyboard and mouse handling changes needed
> 6. the final integrated version ready to paste into my current file

Codex:

> Simplified the title screen by removing the always-visible controls section and replacing it with a modal-style overlay.
>
> The response added:
>
> - `showHowToPlay`
> - button bounds state for click detection
> - `drawHowToPlayOverlay()`
> - `drawStartHowToPlayButton()`
> - `mousePressed()` handling for opening and closing the modal
> - keyboard support using `H` and `ESC`
>
> The overlay contained the controls for `Arrow Keys`, `M`, `L`, and `R`, plus short guidance about green Calm Zones, red noise zones, and managing memory, fatigue, and overload.

5.12 Codex - Button Sizing and Spacing Refinement (Jimin, March 9)

User (Jimin):

> - Remove the small text `Begin the day` underneath the main start button.
> - Remove the line `Press L for Low Sensory Mode` from the start screen.
> - Adjust the sizes of the two buttons so they feel visually balanced, polished, and proportional to each other.
> - The main `Press ENTER to Start` button should remain the primary focal button, but it should not feel oversized or too tall.
> - The secondary `How to Play` button should be resized so it looks intentionally designed in relation to the main start button.

> - Improve the spacing between the title section, main button, and secondary button so the layout feels more intentional and visually balanced.
> - Keep the overall dark, polished, serious, and immersive visual style.
> - Do not redesign the whole screen. Only refine the button sizing, alignment, and spacing.
>
> Please provide:
>
> 1. a short explanation of the visual changes
> 2. the updated code for the start screen layout and button sizing
> 3. any variable changes needed for button dimensions and spacing

Codex:

> Removed the helper line and Low Sensory text from the main title layout, then refined the composition by:
>
> - reducing the panel height
> - introducing shared button dimension constants
> - making the primary button slimmer
> - making the secondary button wider and taller
> - tightening the vertical spacing between the hero text and the button stack
>
> The response also added button sizing constants for width, height, and spacing.

5.13 Codex - Centering Text Blocks Inside the Main Panel (Jimin, March 9)

User (Jimin):

> Can you make all the text blocks to be in center of the panel that is in `drawStartScreen`

Codex:

> Adjusted the layout so the title, subtitle, divider logic, intro lines, and button stack use the panel's own center coordinates rather than fixed screen values.
>
> The response specifically introduced `panelCX` and `panelCY`, recalculated the text anchor positions from those values, and removed the now-unnecessary fixed `START_BUTTON_STACK_Y` placement constant.

5.14 Codex - Aligning Title to the Inner Faded Rectangle (Jimin, March 9)

User (Jimin):

> i want the title and the subtitle located in the center of the faded rectangle instead of the navy rectangle

Codex:

> Moved the title and subtitle from the full panel center to the inner faded hero rectangle by introducing:

> - `heroX`
> - `heroY`
> - `heroW`
> - `heroH`
> - `heroCX`
> - `heroCY`
>
> The response then positioned `titleY` and `subtitleY` relative to that inner rectangle so the highlighted hero area became the actual visual anchor of the start screen.

5.15 Codex - Expanding the March 9 Transcript Appendix (Jimin, March 9)

User (Jimin):

> did you add full transcript for March 09? please add full transcript based on the conversation you and me had

Codex:

> Expanded the March 9 appendix from short summary-only entries into a fuller prompt-by-prompt record covering the stage refactor and title-screen redesign sequence.

5.16 Codex - Collision Cleanup and HUD Simplification (Jimin, March 9)

User (Jimin):

> Make all decorations or obstacles the player should not pass through actually block movement, fix noise zones that overlap obstacles, and remove the subcontext below the goal.

Codex:

> Changed decoration collision so solid props block by default unless marked pass-through, moved overlapping noise zones away from walls and blocking decorations, and removed the extra secondary text line under the HUD goal display.

5.17 Codex - Restoring the Overload Gauge (Jimin, March 9)

User (Jimin):

> why did you remove the overloaded gauge? please revert this only

Codex:

> Restored only the overload gauge call in `drawHUD()` and left the earlier goal-text cleanup intact.

5.18 Codex - Stage 3 Collision and Checkpoint Fix (Jimin, March 9)

User (Jimin):

> In Stage 3 I can still go through an obstacle, and at the second checkpoint in the Break Room the player gets stuck and does not move.

Codex:

> Made the remaining Stage 3 rug obstacle solid and moved the second checkpoint out of the vending-machine collision area so the player no longer gets trapped there.

5.19 Codex - Overload Bar Fill Fit Fix (Jimin, March 9)

User (Jimin):

> In Stage 1 the overload gauge green part does not fit correctly inside the bar rectangle.

Codex:

> Adjusted the overload bar fill drawing so the inner green fill is inset and its corner radius is clamped at very small values, preventing it from poking outside the bar.

5.20 Codex - Title Ambient on Page Open (Jimin, March 9)

User (Jimin):

> I want the `startTitleAmbient` sound to play when I open the web page.

Codex:

> Added the earliest possible title-ambient start on load and also made the first click initialize and resume audio, while noting that browser autoplay policies may still delay actual playback until user interaction.

5.21 Codex - Carrying Title Ambience into Gameplay (Jimin, March 9)

User (Jimin):

> Make the background sound continue after pressing Enter, lower it during gameplay, and keep it from overlapping too heavily with other sounds.

Codex:

> Kept the title ambience running into gameplay at a quieter mix, lowered the regular gameplay ambient layer, and added short ducking so sound effects remain clearer instead of smearing with the background audio.

5.22 Codex - Objective Disappearing Instead of Fading (Jimin, March 9)

User (Jimin):

> For objectives in the stages, I want them removed completely, not faded out when time passes. Then when I press `M`, the objective should come up again.

Codex:

> Removed the faded fallback draw state so objectives now fully disappear when memory time runs out, while keeping the existing `M` recall behavior.

5.23 Codex - Shorter Objective Memory Timing (Jimin, March 9)

User (Jimin):

> Let's make the time shorter for objective disappearance.

Codex:

> Shortened `memoryTimer` values across Stages 1-3 and also reduced the auto-refresh duration after collecting a task so the disappearing-objective mechanic activates more quickly.

5.24 Codex - Showing `hintMemory` Only After Objective Loss (Jimin, March 9)

User (Jimin):

> can you make the `hintMemory` appear when the objective disappears?

Codex:

> Changed the HUD logic so the memory hint now appears only when memory mode is active and the objective is currently hidden.

5.25 Codex - Locating the Emotional Message and Objective Text Code (Jimin, March 9)

User (Jimin):

> I want to adjust the size of the box of `emotionalMessage`. Where does it live in the code?
>
> I wanna adjust the location of the objective text. Where can I find it in the code?

Codex:

> Pointed Jimin to the relevant code locations:
>
> - `drawEmotionMessage()` for the emotional thought box width, height, and corner radius
> - `drawHUD()` for the objective text coordinates
>
> The responses explained which specific numeric values controlled width, height, roundness, and text position.

5.26 Codex - Centering Objective and Hint Text in the Top HUD Bar (Jimin, March 9)

User (Jimin):

> I wanna put the objective and hint texts placed in the middle of the blue rectangle.

Codex:

> Recentered both the objective text and the memory hint inside the top blue HUD bar by introducing shared `hudTextX` and `hudTextY` anchor values.

5.27 Codex - Simplifying the Stage-Complete Transition Screen (Jimin, March 9)

User (Jimin):

> Please redesign and simplify my stage-complete transition screen so it is much easier to understand at a glance.

Codex:

> Rebuilt the transition screen as a compact centered card with stronger hierarchy:
>
> - large gold completion title
> - stage name beneath it
> - one concise `Next · Overload` line
> - a button-styled `Press ENTER to Continue` prompt
>
> The response also introduced a more consistent transition palette and spacing constants.

5.28 Codex - Transition Screen Spacing and Centering Pass (Jimin, March 9)

User (Jimin):

> Make the line located below the stage complete title, make all text gaps the same, and place all of the text in the middle of the rectangle.

Codex:

> Moved the divider line directly below the completion title, normalized the vertical gaps with one stack spacing value, and re-centered the whole transition layout vertically inside the card.

5.29 Codex - Matching Stage 2 Transition to Stage 1 Format (Jimin, March 9)

User (Jimin):

> Please make the same format for the Stage 2 transition, remove the subcontext, and use the same format as the first stage transition page.

Codex:

> Removed the shared transition subcontext/reflection line so Stage 2 uses the exact same cleaner layout pattern as Stage 1.

5.30 Codex - Shorter Stage Intro Text and Centered Intro Banner (Jimin, March 9)

User (Jimin):

> Please make the `introText` for Stage 1, 2, and 3 concise, adjust the rectangle size to fit them all, and make the texts located in the center of the box.

Codex:

> Shortened each stage intro sentence and updated the intro-banner drawing so the box sizes itself to the text while centering both the stage title and intro copy inside it.

5.31 Codex - Repositioning the Stage 3 `Break` Label (Jimin, March 9)

User (Jimin):

> please change the location of the stage 03 star label: `Break`

Codex:

> Added support for per-task label offsets and moved the Stage 3 `Break` label without changing the other task labels.

5.32 Codex - Lowering the Stage 3 Calm Zone Below the Emotional Message (Jimin, March 9)

User (Jimin):

> also the calm zone stage 03, located in the below because it's kinda overlapped with the emotional message.

Codex:

> Moved the Stage 3 calm zone out of the upper HUD/emotional-message band and into a lower clear space in the break-room area.

5.33 Codex - Lowering the Stage 3 Task Marker Below the Emotional Message (Jimin, March 9)

User (Jimin):

> the stars also should be located in the below, it also kinda overlapped with the emotional message

Codex:

> Moved the Stage 3 `Break` task marker downward so it no longer visually overlaps the emotional message area.

5.34 Codex - Moving the Stage 3 Calm Zone and Break Task to an Easier Lower-Middle Area (Jimin, March 9)

User (Jimin):

> okay, it's hard to get into the calm zone. please locate the star and the calm zone in the bottom of the middle part.

Codex:

> Relocated both the Stage 3 break-room calm zone and the `Break` task marker into the lower-middle section so the area is easier to approach and less cramped than the earlier break-room pocket.

5.35 Codex - Shared Lives Across the Full Run (Jimin, March 9)

User (Jimin):

> for the each stages player have 3 lives and it reset when the stage moves on. Do not reset the lives and just keep the lives from the stage 01. So total lives will be 3 not 9

Codex:

> Removed the stage-by-stage life reset so the player now uses one shared three-life pool across the full run, and updated the HUD wording from `Resets left` to `Lives left`.

5.36 Codex - Faster Overall Overload Build-Up (Jimin, March 9)

User (Jimin):

> please make the overall overload guage speed little bit faster.

Codex:

> Added a small global overload-rate multiplier so overload builds slightly faster across the whole game without retuning each stage separately.

5.37 Codex - Adding the Remaining March 9 Transcript Entries to the Process Document (Jimin, March 9)

User (Jimin):

> now please add the full script the conversation I had with you after this conversation

Codex:

> Added the remaining March 9 Codex interaction sequence to this appendix so the documentation reflects the full follow-up iteration history after the title/subtitle alignment exchange.

5.38 Claude Code - Visual Upgrade and Stage Layout Redesign (Jimin, March 9)

User (Jimin):

> Please read the current codes and upgrade the visual design and layout quality of all three gameplay stages in my p5.js game, and also simplify and improve the stage-complete transition screen.

> Context: This is a respectful, empathy-driven game about TBI. The game is meant to help non-disabled players feel cognitive strain, overload, fatigue, and frustration during ordinary daily tasks.
>
> Main goals:
>
> 1. Visually upgrade Stage 1, Stage 2, and Stage 3 so each one feels more intentionally designed
> 2. Make decorations function as actual obstacles when appropriate
> 3. Separate overlapping objects and UI elements so the layout is clearer
> 4. Simplify the stage-complete screen text aggressively
> 5. Improve the overall visual quality of both gameplay stages and the between-stage transition screen
>
> Specific stage design expectations:
>
> - Stage 1 should feel calmer, more spacious, and easier to read
> - Stage 2 should feel more visually stimulating and more pressured, but still readable
> - Stage 3 should feel the most demanding, dense, and stressful, while still fair and navigable
>
> Please also convert suitable decorations into blocking obstacles, adjust positions so nothing overlaps awkwardly, and keep the TBI representation respectful and grounded.

Claude Code:

> Made the following changes across the full codebase:
>
> **Decoration collision system:** Added a `solid` property to decorations and updated `hitsWall()` to check solid decorations as collision obstacles. Solid decorations now get deeper shadows, thicker highlight edges, and a subtle outline so players can visually distinguish them from walk-through decor. Labelled decorations show faint text labels.
>
> **Stage 1 — Home (redesigned):** Open apartment layout with bedroom (left), living room (center), kitchen (upper-right), front hall (lower-right). Solid furniture includes bed, nightstand, TV stand, couch, kitchen counter, table, bookshelf, shoe rack. Walk-through decor: rug, hallway mat. Wide pathways, generous spacing between stars and zones. Warm brown floor tiles (52px grid).
>
> **Stage 2 — Outside / Store (redesigned):** Left third = sidewalk with bus stop, hedge, hydrant obstacles. Right two-thirds = store with 3 aisle shelves, end-cap displays, shopping cart, pharmacy counter. 6 noise zones, 2 calm zones. Cooler teal floor grid (36px), more obstacles, tighter paths.
>
> **Stage 3 — Workplace / Way Home (redesigned):** Office cubicle area (left) with desks, printer, filing cabinet. Break room (center) with sofa, plant, water cooler, vending machine. Transit corridor (right-top) with benches, narrow passages. Final stretch home (right-bottom). 7 noise zones, only 1 calm zone. Dense blue-tinted grid (30px), most obstacles, tightest navigation.
>
> **Stage-complete transition screen:** Replaced multi-paragraph text with a clean centered panel containing: stage completion title, stage name, one short italic reflection line, next stage name, starting overload, and "Press ENTER to Continue."
>
> **Per-stage floor rendering:** Each stage now has a distinct tile grid style (warm wide tiles for home, tighter cooler grid for store, dense institutional grid for workplace).

5.39 Claude Code - Title Screen Ambient Audio (Jimin, March 9)

User (Jimin):

> Add background audio that helps the player feel the sensory strain, pressure, and discomfort often associated with TBI, while keeping it respectful, subtle, and supportive of the gameplay experience rather than overwhelming or distracting. In the title page, and when the game starts, the background sound should stop.

Claude Code:

> Added a 3-layer title screen ambient soundscape using Web Audio API:
>
> - **Layer 1: Low drone** (55 Hz sine, gain 0.025) — persistent pressure sensation, like a dull headache
> - **Layer 2: Faint tinnitus ring** (3800 Hz sine, gain 0.008) — quiet high-frequency tone referencing TBI-associated ringing
> - **Layer 3: Slow pulsing throb** (110 Hz triangle, LFO at 0.3 Hz) — uneasy rhythmic modulation creating subtle tension
>
> All three layers fade in over 2.5 seconds and fade out over 0.6 seconds when pressing ENTER to start. The soundscape restarts when returning to the title screen via R key. Connected `startTitleAmbient()` and `stopTitleAmbient()` to the game state transitions.

5.40 Claude Code - Title Screen TBI-Themed Background Visuals (Jimin, March 9)

User (Jimin):

> For the title page, please improve the visual element in the background related to TBI.

Claude Code:

> Replaced the generic backdrop (blurry ellipses, floating rectangles, drifting dots) with 5 layered TBI-themed visual elements:
>
> 1. **Head silhouette** — Faint breathing oval outline centered behind the panel with an inner "brain region" ellipse, grounding the TBI theme
> 2. **Neural pathway lines** — 9 branching line segments that flicker and break apart on individual cycles, with some segments disappearing to represent damaged connections and small synapse nodes at each joint
> 3. **Drifting thought fragments** — Small rectangular shapes (like scattered words) that float slowly and fade in/out, representing difficulty holding onto thoughts
> 4. **Fading awareness particles** — 50 tiny dots that pulse gently between invisible and barely visible, representing flickering moments of clarity
> 5. **Pressure halo** — Soft warm glow behind the title area representing persistent low-grade discomfort
>
> Everything is very subtle (alpha values 3–22), respectful, and disappears entirely in Low Sensory Mode.

5.41 Claude Code - Win and Lose Screen Redesign (Jimin, March 9)

User (Jimin):

> Please improve the design of the gameover page, reduce the text amount and make it more impactful and eye-catching, as well as the gamesuccess page. The contents in the page should be located in the center of the rectangle.

Claude Code:

> Redesigned both screens with a consistent design language:
>
> **Both screens now share:**
>
> - Dark neutral background matching `COL_BG` instead of harsh solid red/green
> - A centered rounded panel (500px wide) with shadow, semi-transparent body, and subtle accent line
> - All content vertically centered within the panel
> - Gentle ambient particles in the background (rising warm dots for win, slow falling dots for lose)
> - A soft glow behind the panel matching the screen's mood
>
> **Win screen — text reduced from 8 lines to 4:**
>
> - "Day Complete" title, subtitle, overwhelmed count, two short reflection lines, "Press ENTER to Play Again"
>
> **Lose screen — text reduced from 6 lines to 3:**
>
> - "Overwhelmed" title, subtitle, two short compassionate lines, "Press ENTER to Try Again"

---

6. Role-Based Process Evidence

Entry 1

Name: Jimin Kim **Role(s):** Project Setup / Core Game Loop / Game Enhancement **Primary responsibility for this work:** Repository setup, core state machine, stage-based refactor, title screen design, audio system, visual polish, documentation

Goal of Work Session: Iteratively built the game from initial scaffold to a polished 3-stage TBI empathy game. Key phases included: project setup (Mar 3), rubric completion pass (Mar 5), stage-based refactor (Mar 9), and visual/audio polish (Mar 9).

Tools, Resources, or Inputs Used:

- Claude Code (Claude Opus 4.6) via VS Code extension
- Codex (OpenAI GPT-5)
- Assignment rubric and process document template
- Playtesting feedback from Week 6 session
- TBI research references [1][2][3]

GenAI Documentation: GenAI was used extensively for code scaffolding, refactoring, and iterative polish. See Section 4 (entries 4.1, 4.2, 4.7, 4.9, 4.10, 4.11, 4.12, 4.13) and Section 5 (entries 5.9–5.41) for detailed tool disclosure, dates, purposes, and interaction summaries. Human decision points included: choosing the 3-stage structure over the continuous map, directing the emotional tone of transition screens, selecting which decorations should be solid obstacles, and iterating on button sizing and layout alignment through visual feedback.

Summary of Process (Human + Tool): Jimin used Claude Code and Codex as implementation accelerators while making all design direction decisions. The process involved repeated cycles of: prompting for a structural change, reviewing the output, identifying visual or functional issues, and prompting follow-up refinements. For example, the title screen went through 6 iterative prompts (5.9–5.13) before reaching its current state. The stage layouts were redesigned with specific spatial intent (home = spacious, store = pressured, workplace = dense) rather than accepting default generated layouts.

Decision Points & Trade-offs:

1. Chose to keep all code in a single `sketch.js` rather than splitting into modules — prioritized readability for showcase explanations over code organization best practices.
2. Chose data-driven stage definitions over hardcoded per-stage functions — made it easier to tune each stage independently and add new stages if needed.

Verification & Judgement: Tested all three stages for collision issues, checkpoint trapping, overload pacing, and visual overlap. Verified Low Sensory Mode works correctly across all screens. Checked that the title ambient audio stops cleanly when entering gameplay.

Limitations, Dead Ends, or Open Questions:

- Browser autoplay policies may delay title ambient audio until first user interaction
- The single-file structure becomes harder to maintain as the codebase grows
- Some decoration placements may still need fine-tuning after showcase feedback

---

Entry 2

Name: Kaiyang Sun **Role(s):** TBI Mechanics Design & Implementation **Primary responsibility for this work:** Defining which TBI experiences become mechanics, implementing Memory Fade and Sensory Overload

Goal of Work Session: Translated TBI research into beginner-friendly gameplay mechanics that communicate cognitive strain without being literal simulations.

Tools, Resources, or Inputs Used:

- ChatGPT (OpenAI GPT-5)
- TBI research references [1][2][3]
- Teammates (group discussion on mechanic selection)

GenAI Documentation: See Section 4 entries 4.3 and 4.4. ChatGPT was used for brainstorming beginner-friendly mechanic representations and for integrating the mechanics into the existing game structure. The mechanic ideas were simplified and adapted to fit the team's own structure and pacing decisions. No GenAI was used for final implementation tuning.

Summary of Process (Human + Tool): Started by identifying which TBI symptoms could translate into p5.js mechanics without requiring complex systems. Chose Memory Fade (timer + recall) and Sensory Overload (meter + zones) as the two core systems because they are readable, tunable, and directly tied to documented experiences. Later tuned behavior across stages so the mechanics felt progressively harder without becoming random.

Decision Points & Trade-offs: Chose a meter-based overload system over a binary threshold system — the gradual build communicates cumulative strain better than sudden state changes.

Verification & Judgement: Verified mechanics against TBI literature to ensure grounding. Group discussion confirmed the mechanics felt effortful rather than punishing.

Limitations, Dead Ends, or Open Questions: Early versions of Memory Fade used text opacity fading, which was later changed to full disappearance for stronger impact.

Entry 3

Name: Catarina Jin **Role(s):** Level Design / Gameplay Testing **Primary responsibility for this work:** Progression flow, task placement, checkpoint logic, route balancing

Goal of Work Session: Designed level progression so first-time players learn the mechanics before facing the full challenge, and so navigation pressure increases across stages.

Tools, Resources, or Inputs Used:

- ChatGPT (OpenAI GPT-5)
- Playtesting feedback
- Teammates (group discussion on pacing)

GenAI Documentation: See Section 4 entry 4.5. ChatGPT suggested gradual progression and checkpoints. The team manually adjusted layout, task placement, and pacing based on playtesting feedback. No GenAI was used for spatial layout decisions.

Summary of Process (Human + Tool): Identified confusion points during testing where players did not understand the goal or got stuck. Helped rebalance routes so navigation pressure increased over time without becoming random. The key insight from playtesting was that a continuous map flattened the emotional progression — this led to the staged structure decision.

Decision Points & Trade-offs: Chose checkpoint-based respawning over full-stage restart — reduces frustration while still communicating the cost of overload.

Verification & Judgement: Playtesting confirmed that staged progression communicated escalation more clearly than the continuous map.

Limitations, Dead Ends, or Open Questions: Some early checkpoint placements caused player trapping near solid obstacles — these were fixed in later iterations (see 5.19).

Entry 4

Name: Annora Jo **Role(s):** UX/UI Design & Art/Sound / Documentation **Primary responsibility for this work:** Visual tone, readability, accessibility, character expressions, Low Sensory Mode, respectful framing

Goal of Work Session: Shaped the game's visual tone and readability so the presentation remains respectful, grounded, and accessible for a TBI-related showcase context.

Tools, Resources, or Inputs Used:

- ChatGPT (OpenAI GPT-5.2)
- Game Accessibility Guidelines [4]
- Teammates (group discussion on tone)

GenAI Documentation: See Section 4 entries 4.6 and 4.8. ChatGPT was used for UX research on TBI-sensitive design and for fixing facial expression rendering. Design decisions about visual tone, Low Sensory Mode scope, and respectful framing were made through group discussion without GenAI.

Summary of Process (Human + Tool): Focused on ensuring the game avoids horror framing, pity-based messaging, and chaotic punishment systems. Contributed to the distinction between normal and Low Sensory Mode by identifying which effects should be removed for accessibility. Helped ensure character expressions change meaningfully with overload rather than appearing randomly distressed.

Decision Points & Trade-offs: Chose to frame the lose screen around "your brain protecting itself" rather than "game over" language — avoids treating overwhelm as failure while staying honest about the experience.

Verification & Judgement: Reviewed against Game Accessibility Guidelines [4] and group discussion about how TBI survivors might perceive the game at showcase.

Limitations, Dead Ends, or Open Questions: Low Sensory Mode currently removes many effects but does not change the underlying difficulty — a future iteration could also adjust pacing for accessibility.

References

1. Centers for Disease Control and Prevention. 2024. Get the Facts About TBI. *Centers for Disease Control and Prevention*. Retrieved March 9, 2026 from https://www.cdc.gov/traumatic-brain-injury/data-research/facts-stats/
2. Johansson, B., Berglund, P., and Ronnback, L. 2009. Mental fatigue and impaired information processing after mild and moderate traumatic brain injury. *Brain Injury* 23, 13-14, 1027-1040.
3. Lew, H.L., Poole, J.H., Guillory, S.B., Salerno, R.M., Leskin, G., and Sigford, B. 2006. Persistent problems after traumatic brain injury: The need for long-term follow-up and coordinated care. *Journal of Rehabilitation Research and Development* 43, 2, 199-212.
4. Game Accessibility Guidelines. 2012. *Game Accessibility Guidelines*. Retrieved March 9, 2026 from https://gameaccessibilityguidelines.com/
5. Bogost, I. 2007. *Persuasive Games: The Expressive Power of Videogames*. MIT Press, Cambridge, MA.